1. Logger Utility (Method Overloading)

```
class Logger {
    private String logSource;

    public Logger(String logSource) {
        this.logSource = logSource;
    }

    // Overloaded method 1: log with just message
    public void log(String message) {
        System.out.println("[INFO] " + message);
    }

    // Overloaded method 2: log with message and level
    public void log(String message, int level) {
        String levelStr = switch (level) {
            case 1 -> "[ERROR]";
            case 2 -> "[WARNING]";
            case 3 -> "[INFO]";
            case 4 -> "[DEBUG]";
            default -> "[TRACE]";
        };
        System.out.println(levelStr + " " + message);
    }

    // Overloaded method 3: log with message and timestamp
    public void log(String message, String timestamp) {
        System.out.println("[" + timestamp + "] " + message);
    }

    // Additional overload: all parameters combined
    public void log(String message, int level, String timestamp) {
        System.out.println("[" + timestamp + "] [Level " + level + "] " + message);
    }
}

public class LoggerDemo {
    public static void main(String[] args) {
        Logger logger = new Logger("AppLogger");

        System.out.println("=== Logger Utility - Method Overloading Demo ===\n");

        // Calling different overloaded versions
        logger.log("System started");
        logger.log("Database connection failed", 1);
        logger.log("User logged in", "2026-04-21 10:30:00");
        logger.log("Processing request", 4, "2026-04-21 10:30:05");

    }
```

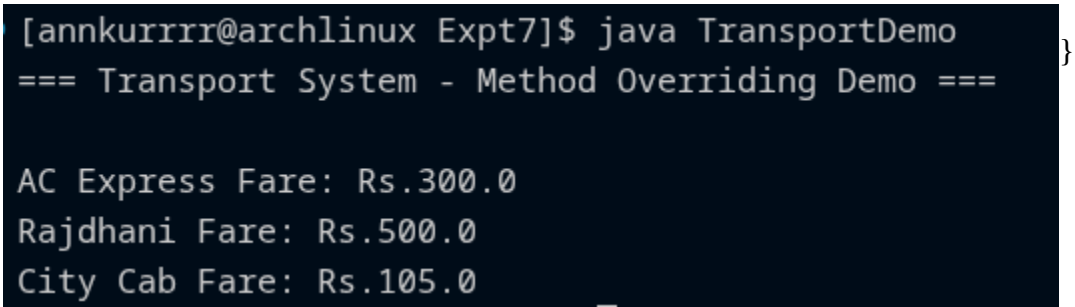
}

```
[annkurrrr@archlinux Expt7]$ java LoggerDemo
=== Logger Utility - Method Overloading Demo ===

[INFO] System started
[ERROR] Database connection failed
[2026-04-21 10:30:00] User logged in
[2026-04-21 10:30:05] [Level 4] Processing request
```

2. Transport System (Method Overriding)

```
class Transport {
    protected String name;
    protected double baseFare;

    public Transport(String name, double baseFare) {
        this.name = name;
        this.baseFare = baseFare;
    }

    // Method to be overridden by subclasses
    public double fare() {
        return baseFare;
    }

    public String getName() {
        return name;
    }
}

class Bus extends Transport {
    private int distance;
    private double perKmRate;

    public Bus(String name, double baseFare, int distance, double perKmRate) {
        super(name, baseFare);
        this.distance = distance;
        this.perKmRate = perKmRate;
    }

    @Override
    public double fare() {
        return baseFare + (distance * perKmRate);
    }
}

class Train extends Transport {
```

```
    private String coachType;
    private double coachMultiplier;

    public Train(String name, double baseFare, String coachType, double coachMultiplier) {
        super(name, baseFare);
        this.coachType = coachType;
        this.coachMultiplier = coachMultiplier;
    }

    @Override
    public double fare() {
        return baseFare * coachMultiplier;
    }
}

class Taxi extends Transport {
    private double waitingTime;
    private double waitingRate;

    public Taxi(String name, double baseFare, double waitingTime, double waitingRate) {
        super(name, baseFare);
        this.waitingTime = waitingTime;
        this.waitingRate = waitingRate;
    }

    @Override
    public double fare() {
        return baseFare + (waitingTime * waitingRate);
    }
}

public class TransportDemo {
    public static void main(String[] args) {
        System.out.println("=== Transport System - Method Overriding Demo ===\n");

        // Creating objects
        Transport bus = new Bus("AC Express", 50.0, 100, 2.5);
        Transport train = new Train("Rajdhani", 200.0, "AC", 2.5);
        Transport taxi = new Taxi("City Cab", 30.0, 15, 5.0);

        // Displaying fare calculations
        System.out.println(bus.getName() + " Fare: Rs." + bus.fare());
        System.out.println(train.getName() + " Fare: Rs." + train.fare());
        System.out.println(taxi.getName() + " Fare: Rs." + taxi.fare());
```




```
    }
}
```

3. Game Characters (Dynamic Method Dispatch)

```
class Character {
    protected String name;
    protected int health;
    protected int attackPower;

    public Character(String name, int health, int attackPower) {
        this.name = name;
        this.health = health;
        this.attackPower = attackPower;
    }

    // Method to be dynamically dispatched
    public void attack() {
        System.out.println(name + " performs a basic attack!");
    }

    public void takeDamage(int damage) {
        health -= damage;
        System.out.println(name + " takes " + damage + " damage. Health: " + health);
    }

    public String getName() {
        return name;
    }

    public int getHealth() {
        return health;
    }
}

class Warrior extends Character {
    private int ragePoints;

    public Warrior(String name, int health, int attackPower, int ragePoints) {
        super(name, health, attackPower);
        this.ragePoints = ragePoints;
    }

    @Override
    public void attack() {
        System.out.println(name + " swings sword with " + ragePoints + " rage points!");
        System.out.println("Warrior deals " + (attackPower * 2) + " damage!");
```

```
    }

    public void specialAttack() {
        System.out.println(name + " uses BERSERKER RAGE! Damage doubled!");
    }
}

class Mage extends Character {
    private int manaPoints;

    public Mage(String name, int health, int attackPower, int manaPoints) {
        super(name, health, attackPower);
        this.manaPoints = manaPoints;
    }

    @Override
    public void attack() {
        System.out.println(name + " casts a spell using " + manaPoints + " mana points!");
        System.out.println("Mage deals " + (attackPower * 3) + " magical damage!");
    }

    public void specialAttack() {
        System.out.println(name + " casts FIREBALL! Area damage dealt!");
    }
}

class Archer extends Character {
    private int arrows;

    public Archer(String name, int health, int attackPower, int arrows) {
        super(name, health, attackPower);
        this.arrows = arrows;
    }

    @Override
    public void attack() {
        System.out.println(name + " shoots arrow! " + arrows + " arrows remaining!");
        System.out.println("Archer deals " + (attackPower * 2) + " piercing damage!");
    }

    public void specialAttack() {
        System.out.println(name + " uses MULTI-SHOT! Fires 5 arrows at once!");
    }
}

public class GameCharactersDemo {
    public static void main(String[] args) {
        System.out.println("=== Game Characters - Dynamic Method Dispatch Demo ===\n");
```

```
        // Dynamic method dispatch: parent reference, child objects
        Character[] party = new Character[3];
        party[0] = new Warrior("Thorin", 150, 25, 80);
        party[1] = new Mage("Gandalf", 80, 35, 100);
        party[2] = new Archer("Legolas", 100, 30, 20);

        // Runtime polymorphism - which attack() is called depends on actual object
        for (Character c : party) {
            System.out.println("--- " + c.getName() + " (HP: " + c.getHealth() + ") ---");
            c.attack();  // Dynamic binding determines which attack() to call
            System.out.println();
        }

        // Type casting to access subclass-specific methods
        ((Warrior) party[0]).specialAttack();
        ((Mage) party[1]).specialAttack();
        ((Archer) party[2]).specialAttack();

    }
}
```

```
[annkurrrr@archlinux Expt7]$ java GameCharactersDemo
=== Game Characters - Dynamic Method Dispatch Demo ===

--- Thorin (HP: 150) ---
Thorin swings sword with 80 rage points!
Warrior deals 50 damage!

--- Gandalf (HP: 80) ---
Gandalf casts a spell using 100 mana points!
Mage deals 105 magical damage!

--- Legolas (HP: 100) ---
Legolas shoots arrow! 20 arrows remaining!
Archer deals 60 piercing damage!

Thorin uses BERSERKER RAGE! Damage doubled!
Gandalf casts FIREBALL! Area damage dealt!
Legolas uses MULTI-SHOT! Fires 5 arrows at once!
```

4. Shape Demo (Interface Polymorphism)

```
interface Shape {
    void draw();
    double area();
    double perimeter();
```

```
}

class Circle implements Shape {
    private double radius;
    private String color;

    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " Circle with radius " + radius);
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * radius;
    }

    public double getRadius() {
        return radius;
    }
}

class Square implements Shape {
    private double side;
    private String color;

    public Square(double side, String color) {
        this.side = side;
        this.color = color;
    }

    @Override
    public void draw() {
        System.out.println("Drawing " + color + " Square with side " + side);
    }

    @Override
    public double area() {
        return side * side;
    }
```

```
    @Override
    public double perimeter() {
        return 4 * side;
    }

    public double getSide() {
        return side;
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        System.out.println("=== Interface Polymorphism - Shape Demo ===\n");

        // Interface reference holding implementation objects
        Shape[] shapes = new Shape[2];
        shapes[0] = new Circle(5.0, "Red");
        shapes[1] = new Square(4.0, "Blue");

        // Polymorphic behavior through interface
        for (Shape shape : shapes) {
            shape.draw();
            System.out.printf("  Area: %.2f sq units%n", shape.area());
            System.out.printf("  Perimeter: %.2f units%n", shape.perimeter());
            System.out.println();
        }

        // Individual access with type casting
        Circle circle = (Circle) shapes[0];
        Square square = (Square) shapes[1];

        System.out.println("=== Shape Properties ===");
        System.out.println("Circle radius: " + circle.getRadius());
        System.out.println("Square side: " + square.getSide());

    }
}
```

```
[annkurrrr@archlinux Expt7]$ java ShapeDemo
=== Interface Polymorphism - Shape Demo ===

Drawing Red Circle with radius 5.0
  Area: 78.54 sq units
  Perimeter: 31.42 units

Drawing Blue Square with side 4.0
  Area: 16.00 sq units
  Perimeter: 16.00 units

=== Shape Properties ===
Circle radius: 5.0
Square side: 4.0
```